

11. JavaScript – syntaxe, datové typy, funkce, promise

Základní syntaxe

- Na konci řádků středníky
- Proměnné: let, const, var
- Funkční operátory:
 - Aritmetické: +, -, *, /, %, **
 - Porovnávací: == (volná), === (striktní), !=, !==, >, <, >=, <=
 - Logické: &&, ||, !
- Cykly: For, While, Do ... while, for ... of, for ... in
- Podmínky: if, else if, else

```
for (let i = 0; i < 5; i++) {  
  if (i % 2 === 0) {  
    console.log(i + ' is even');  
  } else {  
    console.log(i + ' is odd');  
  }  
}
```

Proměnné

- Let – může být později přepsána, platná pouze v daném “pohledu”/bloku (scope)
- Var – považován za outdated, podobný jako let ale platný v celé funkci, narozdíl od let a const se vytáhne na začátek funkce (tzn. i když je deklarovaný na konci tak lze použít i na začátku = hoisting)
- Const – později nesmí být přepsána (konstantní hodnota), platná v scope

Datové typy

- Primitivní: String, Number, Boolean, Null, Undefined, Symbol, BigInt
- Složené: Object (i Array, Function)

Funkce

- Typy:
 - Klasická – zápis normální
 - Anonymní – zápis normální ale pomocí uložení do proměnné
 - Arrow – zápis do proměnné bez klíč. slova function ale s šipkou, implicitní return = pokud je na jeden řádek vrací automaticky co na něm je, arrow funkce nemá vlastní this, přebírá ho z okolního scope, proto je skvělá v metodách tříd, ale problematická v objektových literálech
- Fungují obdobně jako v Jave nebo C#

```
// Klasická funkce
function greet(name) {
  return 'Hello, ' + name;
}

// Arrow funkce
const greet = (name) => `Hello, ${name}`;
greet("Ječná");

// Anonymní funkce
var greet = function (name) {
  console.log(`Hello, ${name}`);
};

greet("Ječná");
```

Promise

- pro zjednodušení práce s asynchronním kódem
- řeší Callback hell (noření do funkcí)
- lze psát logiku v sekvenci s jednotným způsobem ošetření chyb
- doslova slib, budoucí výsledek async operace
- tvorba pomocí `new Promise(executor)`, `executor` je funkce s param `resolve` a `reject`
- Při úspěchu zavolá `resolve(value)`
- Při chybě zavolá `reject(error)`

- Lze na to navázat *.then*, *.catch*, *.finally* = co se má stat potom
- Navázané věci se dají řetězit
- Stav: pending, fulfilled, rejected
- Promise.all(pole promis) = vrátí pole hodnot všech vložených promis, vrátí se po dokončení všech promis v poli
- Promise.race(pole promis) = vrátí to co vrací promis, která se splní jako první
- Promise.allSettled(pole promis) = vrací pole, se statusem (fulfilled, rejected) a na základě toho s value když splněna nebo s reason když ne

```
fetchData() // vrací Promise
  .then(data => process(data)) // zpracuj získaná data
  .then(result => save(result)) // další asynchronní krok
  .then(() => console.log("Hotovo"))
  .catch(error => console.error("Chyba:", error));
```